

# Chapter 4.6: Pipelined Datapath and Control

Detailed Lecture Notes with Visuals

## Contents

|          |                                                                  |          |
|----------|------------------------------------------------------------------|----------|
| <b>1</b> | <b>Introduction: Pipelining the Single-Cycle Datapath</b>        | <b>2</b> |
| 1.1      | The Five Stages . . . . .                                        | 2        |
| 1.2      | Flow of Data . . . . .                                           | 2        |
| <b>2</b> | <b>Pipeline Registers</b>                                        | <b>2</b> |
| 2.1      | Register Sizing . . . . .                                        | 2        |
| <b>3</b> | <b>Instruction Walkthroughs</b>                                  | <b>4</b> |
| 3.1      | 1. Load Instruction ( <b>ld</b> ) . . . . .                      | 4        |
| 3.2      | 2. Store Instruction ( <b>sd</b> ) . . . . .                     | 4        |
| 3.3      | 3. R-Type Instruction (e.g., <b>add</b> , <b>sub</b> ) . . . . . | 4        |
| <b>4</b> | <b>The Design “Bug” &amp; Structural Hazard</b>                  | <b>4</b> |
| <b>5</b> | <b>Visualizing Pipelines</b>                                     | <b>5</b> |
| 5.1      | 1. Multi-Clock-Cycle Diagram (Traditional Style) . . . . .       | 5        |
| 5.2      | 2. Single-Clock-Cycle Diagram (Vertical Slice) . . . . .         | 5        |
| <b>6</b> | <b>Pipelined Control</b>                                         | <b>5</b> |
| 6.1      | Control Signal Propagation . . . . .                             | 6        |
| <b>7</b> | <b>Check Yourself (from text)</b>                                | <b>6</b> |
| <b>8</b> | <b>Flashcards</b>                                                | <b>7</b> |

# 1 Introduction: Pipelining the Single-Cycle Datapath

“There is less in this than meets the eye.” – Tallulah Bankhead

The division of an instruction into five stages means a five-stage pipeline. Consequently, up to **five instructions** will be in execution during any single clock cycle.

## 1.1 The Five Stages

We separate the datapath into five pieces, named after the stage of execution:

1. **IF**: Instruction Fetch
2. **ID**: Instruction Decode and Register File Read
3. **EX**: Execution or Address Calculation
4. **MEM**: Data Memory Access
5. **WB**: Write Back

## 1.2 Flow of Data

Instructions and data generally move from **left to right** through these five stages. They get “cleaner, drier, and more organized” as they move through the line (Laundry Analogy).

### Exceptions to Left-to-Right Flow

There are two critical exceptions where data flows **right-to-left**:

1. **Write-Back Stage (WB)**: Places the result back into the Register File in the middle of the datapath (Stage 5 → Stage 2). This causes *Data Hazards*.
2. **PC Update**: Selecting the next PC value (Branch address from MEM → IF). This causes *Control Hazards*.

# 2 Pipeline Registers

To share a single datapath among multiple instructions without them interfering, we must place **Pipeline Registers** between each stage. These act like “baskets” in the laundry analogy, holding data for the next stage.

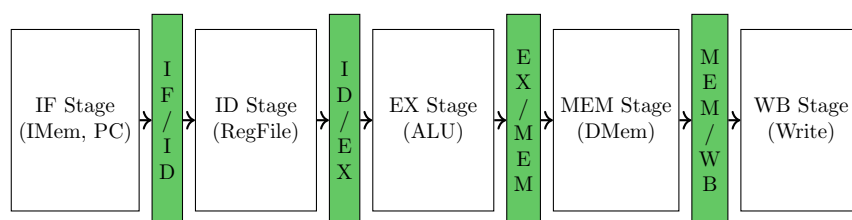


Figure 1: Pipelined Datapath with Pipeline Registers (Highlighted in Green)

## 2.1 Register Sizing

These registers must be wide enough to store **ALL** data needed for subsequent stages.

- **IF/ID (96 bits)**: Holds 32-bit Instruction + 64-bit PC.
- **ID/EX (256 bits)**: Holds Register Values, Immediate, PC, etc.
- **EX/MEM (193 bits)**: Holds ALU Result, Write Data, Dest Reg, etc.
- **MEM/WB (128 bits)**: Holds Memory Data, ALU Result, Dest Reg.

**Note**

There is **no pipeline register** at the end of WB because the state (Register File, Memory, PC) is updated directly.

### 3 Instruction Walkthroughs

#### 3.1 1. Load Instruction (ld)

Active in all 5 stages.

- **IF:** Fetch instruction, increment PC. Save to IF/ID.
- **ID:** Read registers, sign-extend immediate. Save to ID/EX.
- **EX:** ALU adds register + immediate (address calculation). Save to EX/MEM.
- **MEM:** Read data memory using address. Save to MEM/WB.
- **WB:** Write loaded data into Register File.

#### 3.2 2. Store Instruction (sd)

- **IF & ID:** Same as Load.
- **EX:** Calculate address. **Crucially**, we must also pass the value to be stored (read in ID) from ID/EX to EX/MEM.
- **MEM:** Write data to memory.
- **WB: Idle.** Nothing happens.

#### 3.3 3. R-Type Instruction (e.g., add, sub)

Although 4.6.txt primarily uses Load/Store to illustrate the datapath, R-Types are simpler:

- **IF & ID:** Same as Load.
- **EX:** ALU performs operation. Result to EX/MEM.
- **MEM: Idle.** Result passes through.
- **WB:** Result written to Register File.

## 4 The Design “Bug” & Structural Hazard

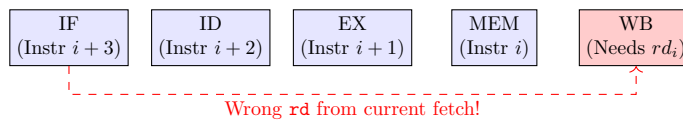
### The Bug: Destination Register

In the native breakdown, the instruction is decoded in **ID**, identifying the destination register (**rd**). However, the write occurs in **WB**.

**The Problem:** By the time the Load instruction reaches WB (5 cycles later), the IF stage is fetching a *different* instruction. WB would use the wrong register bits!

**The Fix:** We must **preserve the destination register number** by passing it along the pipeline registers.

## A. The Bug (Wrong)



## B. The Fix (Correct)



Figure 2: Visualizing the Bug (Top) and the Fix (Bottom).

## 5 Visualizing Pipelines

### 5.1 1. Multi-Clock-Cycle Diagram (Traditional Style)

Shows execution over time with detailed stage activities (similar to Figure 4.42).

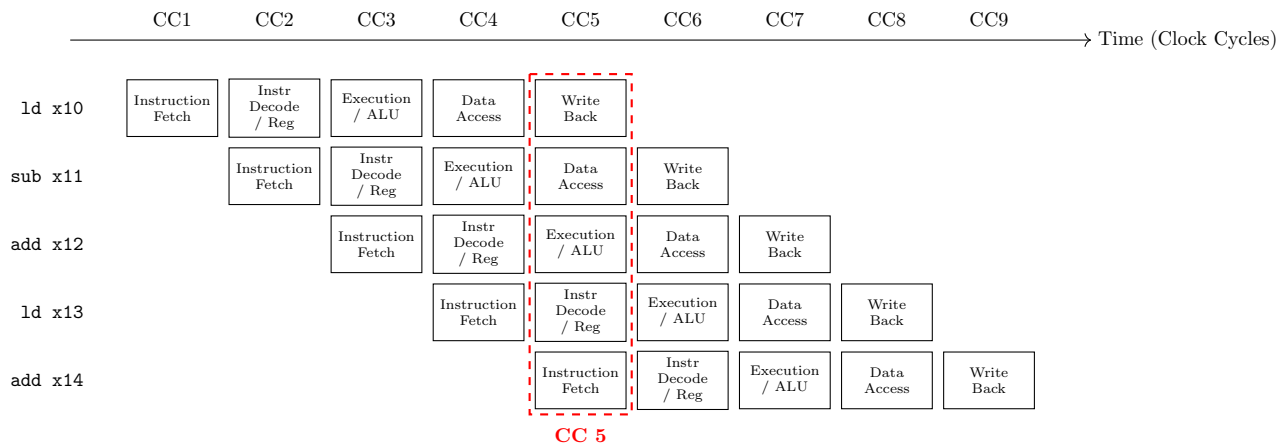
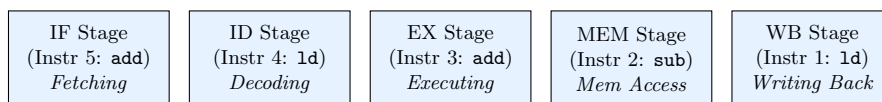


Figure 3: Traditional Multiple-Clock-Cycle Pipeline Diagram. In Cycle 5, each instruction is in a different stage.

### 5.2 2. Single-Clock-Cycle Diagram (Vertical Slice)

Represents the state of the processor at a specific instant (e.g., Cycle 5).



At Cycle 5, the hardware is fully utilized, handling 5 different instructions simultaneously without conflict (assuming no hazards).

Figure 4: Single-Clock-Cycle Diagram: A snapshot of the processor state at Cycle 5.

## 6 Pipelined Control

Control lines are just like single-cycle, but applied at different times. We group them by stage and pass them through pipeline registers.

| Stage | Control Lines             | Function            |
|-------|---------------------------|---------------------|
| IF    | (None)                    | standard read/write |
| ID    | (None)                    | Fixed decoding      |
| EX    | ALUOp, ALUSrc             | Execution Control   |
| MEM   | Branch, MemRead, MemWrite | Memory/Branching    |
| WB    | MemtoReg, RegWrite        | Register Write      |

### 6.1 Control Signal Propagation

Control signals are generated in the **ID stage** but used in later stages (EX, MEM, WB). To ensure they arrive at the correct time, they are passed through the pipeline registers:

1. **ID/EX Register:** Holds all control signals for EX, MEM, and WB.
2. **EX/MEM Register:** Passes MEM and WB signals.
3. **MEM/WB Register:** Passes WB signals.

## 7 Check Yourself (from text)

1. **True/False:** allowing instructions to take fewer stages always increases performance?  
*False. Throughput is determined by the clock cycle time (slowest stage), not individual instruction latency.*
2. **True/False:** Pipeline stages affect latency, not throughput?  
*True.*
3. **True/False:** You can make branches take fewer cycles?  
*True, but it complicates control.*
4. **True/False:** Making the pipeline longer (more stages) can improve performance?  
*True, if it allows for a shorter clock cycle (higher frequency).*

## 8 Flashcards

1. **Q: What is the first stage of the five-stage pipeline, and what is its abbreviation?**  
A: The first stage is Instruction Fetch, abbreviated as IF.
2. **Q: What is the second stage of the five-stage pipeline, and what is its abbreviation?**  
A: The second stage is Instruction Decode and register file read, abbreviated as ID.
3. **Q: What is the third stage of the five-stage pipeline, and what is its abbreviation?**  
A: The third stage is Execution or address calculation, abbreviated as EX.
4. **Q: What is the fourth stage of the five-stage pipeline, and what is its abbreviation?**  
A: The fourth stage is Data memory access, abbreviated as MEM.
5. **Q: What is the fifth stage of the five-stage pipeline, and what is its abbreviation?**  
A: The fifth stage is Write back, abbreviated as WB.
6. **Q: In a five-stage pipeline, up to how many instructions can be in execution during a single clock cycle?**  
A: Up to five instructions can be in execution during any single clock cycle.
7. **Q: What is the general direction of flow for instructions and data through the pipeline stages?**  
A: Instructions and data generally move from left to right through the five stages.
8. **Q: What is the first exception to the left-to-right flow of data in the pipelined datapath?**  
A: The write-back stage, which sends the result back to the register file in the middle of the datapath.
9. **Q: What is the second exception to the left-to-right flow of data in the pipelined datapath?**  
A: The selection of the next PC value, which chooses between the incremented PC and a branch address from the MEM stage.
10. **Q: The right-to-left data flow of the write-back stage can lead to what type of hazard?**  
A: This flow can lead to data hazards.
11. **Q: The right-to-left data flow for selecting the next PC value can lead to what type of hazard?**  
A: This flow can lead to control hazards.
12. **Q: To share a single datapath among multiple instructions in a pipeline, what hardware components must be added between stages?**  
A: Pipeline registers must be added to hold data between stages.
13. **Q: What is the naming convention for pipeline registers?**  
A: They are named for the two stages they separate (e.g., the register between IF and ID is named IF/ID).
14. **Q: Why is there no pipeline register at the end of the write-back (WB) stage?**  
A: It is redundant because all instructions must update some processor state (register file, memory, or PC) which already holds the result.
15. **Q: Which component of the processor's visible architectural state can be thought of as a pipeline register that feeds the IF stage?**  
A: The Program Counter (PC) can be thought of as a pipeline register feeding the IF stage.
16. **Q: What two pieces of information are placed into the IF/ID pipeline register during the Instruction Fetch stage?**  
A: The fetched instruction and the incremented PC address are placed in the IF/ID register.
17. **Q: During the ID stage of a load instruction, what values are stored in the ID/EX pipeline register?**  
A: The two read register values, the sign-extended immediate value, and the PC address are stored in the ID/EX register.
18. **Q: What calculation is performed by the ALU during the EX stage of a load ('ld') instruction?**  
A: The ALU adds the contents of a register and the sign-extended immediate to calculate the effective memory address.

19. **Q: During the MEM stage of a load ('ld') instruction, data is read from memory and placed into which pipeline register?**  
A: The data read from memory is placed into the MEM/WB pipeline register.
20. **Q: What is the final action performed during the WB stage of a load ('ld') instruction?**  
A: Data is read from the MEM/WB pipeline register and written into the register file.
21. **Q: For a store instruction, what is passed from the ID/EX pipeline register to the EX/MEM pipeline register to be used in the MEM stage?**  
A: The register data to be stored is passed into the EX/MEM pipeline register.
22. **Q: What is the primary action during the MEM stage of a store ('sd') instruction?**  
A: The data from the EX/MEM pipeline register is written into data memory.
23. **Q: What happens during the write-back (WB) stage for a store ('sd') instruction?**  
A: Nothing happens in the write-back stage for a store instruction.
24. **Q: To prevent a structural hazard, each logical component of the datapath (e.g., ALU, data memory) can be used within how many pipeline stages?**  
A: Each logical component can be used only within a single pipeline stage.
25. **Q: What was the design bug in the initial pipelined datapath concerning the load ('ld') instruction?**  
A: The write register number was not passed down the pipeline, so the wrong instruction would supply it during the WB stage.
26. **Q: How was the design bug for the load ('ld') instruction fixed?**  
A: The destination register number is passed from the ID stage through the ID/EX, EX/MEM, and MEM/WB pipeline registers for use in the WB stage.
27. **Q: What is the key difference between a multiple-clock-cycle pipeline diagram and a single-clock-cycle pipeline diagram?**  
A: A multiple-clock-cycle diagram shows instruction execution over time, while a single-clock-cycle diagram shows the state of the entire datapath at one specific clock cycle.
28. **Q: A single-clock-cycle diagram represents a \_\_\_\_\_ slice of a multiple-clock-cycle diagram.**  
A: vertical
29. **Q: Why does allowing some instructions to take fewer cycles not necessarily improve pipeline throughput?**  
A: Throughput is determined by the clock cycle, not the latency (number of stages) of individual instructions.
30. **Q: Instead of making instructions take fewer cycles, what alternative approach could improve pipeline performance?**  
A: Making the pipeline longer so instructions take more, but shorter, clock cycles could improve performance.
31. **Q: Control signals for the EX, MEM, and WB stages are created during which pipeline stage?**  
A: The control signals are created during the instruction decode (ID) stage.
32. **Q: How are control signals transported to the correct pipeline stage for execution?**  
A: They are passed down the pipeline by extending the pipeline registers (ID/EX, EX/MEM, MEM/WB) to include control bits.
33. **Q: Which two control lines are set in the Execution/address calculation (EX) stage?**  
A: The ALUOp and ALUSrc control lines are set in the EX stage.
34. **Q: Which three control lines are set in the Memory access (MEM) stage?**  
A: The Branch, MemRead, and MemWrite control lines are set in the MEM stage.
35. **Q: Which two control lines are set in the Write-back (WB) stage?**  
A: The MemtoReg and RegWrite control lines are set in the WB stage.
36. **Q: The control signals created in the ID stage are first placed into which pipeline register?**  
A: They are placed into the ID/EX pipeline register.



37. **Q: Why is an instruction passed through a pipeline stage even if there is nothing for it to do in that stage?**  
A: This is done because later instructions are already progressing at the maximum rate, and there is no way to accelerate them.
38. **Q: What information must be preserved and passed down the pipeline for a load instruction to function correctly?**  
A: The destination register number must be preserved and passed through the pipeline registers to the WB stage.
39. **Q: In the stylized datapath diagrams, the register file is broken into two logical parts to represent its dual use in which two stages?**  
A: It is used for register reads during the ID stage and register writes during the WB stage.
40. **Q: What is the purpose of the ALUSrc control signal in the EX stage?**  
A: It selects whether the second ALU input is from a register (Read data 2) or the sign-extended immediate.
41. **Q: What is the purpose of the MemtoReg control signal in the WB stage?**  
A: It decides whether the value written to the register file comes from the ALU result or from data memory.
42. **Q: What is the primary activity in the Instruction Fetch (IF) stage of a pipeline?**  
A: Reading an instruction from memory using the address in the Program Counter (PC).
43. **Q: What are the two main activities in the Instruction Decode (ID) stage of a pipeline?**  
A: Decoding the instruction and reading the contents of the specified registers from the register file.
44. **Q: What is the primary activity in the Execution (EX) stage of a pipeline?**  
A: Performing an operation in the ALU, such as an arithmetic calculation or an address calculation.
45. **Q: What is the primary activity in the Data Memory Access (MEM) stage of a pipeline?**  
A: Reading from or writing to the data memory.
46. **Q: What is the primary activity in the Write Back (WB) stage of a pipeline?**  
A: Writing the result of the instruction (from the ALU or memory) back into the register file.
47. **Q: In a five-stage pipeline, how many instructions can be in execution during a single clock cycle?**  
A: Up to five instructions.
48. **Q: What is the general direction of data and instruction flow through the pipelined datapath stages?**  
A: From left to right, from the IF stage to the WB stage.
49. **Q: What is the first exception to the left-to-right data flow in a pipelined datapath?**  
A: The write-back stage, which sends the result back to the register file located in the middle of the datapath.
50. **Q: What is the second exception to the left-to-right data flow in a pipelined datapath?**  
A: The selection of the next PC value, which can choose a branch address from the MEM stage.
51. **Q: The right-to-left data flow from the write-back stage can lead to what type of hazard?**  
A: Data hazards.
52. **Q: The right-to-left data flow for PC selection (branching) can lead to what type of hazard?**  
A: Control hazards.
53. **Q: What hardware components are added between the stages of a datapath to implement pipelining?**  
A: Pipeline registers.
54. **Q: The pipeline register located between the Instruction Fetch and Instruction Decode stages is named \_\_\_\_.**  
A: IF/ID.

55. **Q: What is the purpose of pipeline registers in a pipelined datapath?**  
A: To hold the data and control information for an instruction as it moves from one stage to the next, allowing stages to be shared by different instructions simultaneously.
56. **Q: In the IF stage of a 'ld' instruction, what happens to the PC value?**  
A: It is incremented by 4 and written back into the PC for the next cycle, and a copy is saved in the IF/ID pipeline register.
57. **Q: In the ID stage of a 'ld' instruction, what values are stored in the ID/EX pipeline register?**  
A: The values from two read registers, the sign-extended immediate value, and the PC address.
58. **Q: What calculation is performed in the EX stage of a 'ld' instruction?**  
A: The ALU adds the contents of a register to the sign-extended immediate value to calculate the memory address.
59. **Q: What happens in the MEM stage of a 'ld' instruction?**  
A: The data memory is read using the address from the EX/MEM pipeline register, and the loaded data is placed in the MEM/WB pipeline register.
60. **Q: In the WB stage of a 'ld' instruction, data is read from the \_\_\_\_\_ pipeline register and written into the register file.**  
A: MEM/WB
61. **Q: How does a 'store' instruction make the data to be stored available during the MEM stage?**  
A: The data is read from the register file in the ID stage and passed through the ID/EX and EX/MEM pipeline registers.
62. **Q: What happens in the Write-Back (WB) stage for a 'store' instruction?**  
A: Nothing happens in the WB stage for a store instruction, as the result was already written to memory in the MEM stage.
63. **Q: To avoid structural hazards, each logical component of the datapath, such as the ALU, can be used only within a single \_\_\_\_\_.**  
A: pipeline stage
64. **Q: What was the original design bug in handling the 'ld' instruction's write-back stage?**  
A: The write register number was being supplied by the instruction in the IF/ID pipeline register, not the 'ld' instruction that was actually in the WB stage.
65. **Q: How was the 'ld' instruction bug fixed in the pipelined datapath design?**  
A: The destination register number is passed from the ID stage through the ID/EX, EX/MEM, and MEM/WB pipeline registers to be used in the WB stage.
66. **Q: What type of pipeline diagram shows the complete execution of multiple instructions over time, with clock cycles on one axis and instructions on the other?**  
A: A multiple-clock-cycle pipeline diagram.
67. **Q: What type of pipeline diagram shows the state of the entire datapath during a single clock cycle?**  
A: A single-clock-cycle pipeline diagram.
68. **Q: A single-clock-cycle diagram represents a vertical slice through what other type of diagram?**  
A: A multiple-clock-cycle diagram.
69. **Q: How are control signals for the EX, MEM, and WB stages passed down the pipeline?**  
A: They are generated in the ID stage and then stored in the control portions of the pipeline registers (ID/EX, EX/MEM, MEM/WB).
70. **Q: Which two control signals are set during the Execution/address calculation (EX) stage?**  
A: ALUOp and ALUSrc.
71. **Q: Which three control signals are set during the Memory access (MEM) stage?**  
A: Branch, MemRead, and MemWrite.

72. **Q: Which two control signals are set during the Write-back (WB) stage?**  
A: MemtoReg and RegWrite.
73. **Q: Why do the IF and ID stages not require special control signals to be passed to them?**  
A: Their primary operations (reading instruction memory, writing the PC, reading registers) are always asserted or are determined by fixed fields in the instruction format.
74. **Q: In the corrected datapath, the write register number for the WB stage comes from which pipeline register?**  
A: The MEM/WB pipeline register.
75. **Q: What information must be stored in the IF/ID pipeline register for a 'beq' instruction to potentially work later on?**  
A: The incremented PC address.
76. **Q: During the EX stage of a 'store' instruction, what two values are placed into the EX/MEM pipeline register?**  
A: The calculated effective address and the second register value (the data to be stored).
77. **Q: According to the 'Check Yourself' section, is it true that allowing some instructions to take fewer stages always increases pipeline performance?**  
A: No, because pipeline throughput is determined by the clock cycle time, not the latency of individual instructions.
78. **Q: The number of pipeline stages per instruction primarily affects an instruction's \_\_\_\_\_, not the pipeline's throughput.**  
A: latency
79. **Q: According to the 'Check Yourself' section, what is a potential benefit of making a pipeline longer (more stages)?**  
A: A longer pipeline can allow for a shorter clock cycle time, which could improve overall performance (throughput).
80. **Q: The IF/ID pipeline register must be wide enough to hold both the 32-bit instruction and the incremented \_\_\_\_ address.**  
A: 64-bit PC
81. **Q: The control logic generates control signals for the EX, MEM, and WB stages during which pipeline stage?**  
A: The Instruction Decode (ID) stage.
82. **Q: The control line \_\_\_\_\_ determines whether the ALU gets its second operand from the register file or the sign-extended immediate field.**  
A: ALUSrc
83. **Q: The control line \_\_\_\_\_ determines whether the value written to the register file comes from the ALU result or from data memory.**  
A: MemtoReg
84. **Q: The \_\_\_\_\_ control signal must be asserted to enable writing to the register file in the WB stage.**  
A: RegWrite
85. **Q: What two signals must both be asserted for the 'PCSrc' multiplexor to select the branch target address?**  
A: The 'Branch' signal (from control) and the 'Zero' signal (from the ALU).